Acoplamento e Coesão

Introdução

- É possível implementar uma aplicação e fazer com que ela realize sua tarefa com classes mal projetadas.
- Apenas executar uma aplicação concluída não indica se está bem estruturada ou não.
- Problemas surgem quando programador de manutenção precisa fazer alterações em uma aplicação existente.

Introdução

- Ex:
  - Corrigir falha
  - Adicionar nova funcionalidade

  - Uma tarefa que poderia ser fácil e óbvia com classes bem projetadas pode ser muito difícil e envolver muito trabalho caso contrário.

Introdução

- Em aplicações maiores esse efeito já aparece durante a implementação
- De forma geral empresas mantêm, estendem e vendem uma aplicação durante anos
  - Não dá para arcar com o custo de um código mal estruturado

Introdução

- Exemplo:
  - Jogo world-of-zuul
  - Implementação simples de um jogo de aventura baseado em texto
  - Para quem quiser saber mais, o jogo foi desenvolvido na década de 70 por Will Crowther e expandido por Don Woods
    - Jogo original também é conhecido como Colossal Cave Adventure

Exemplo

- Zuul-bad
  - Design ruim
  - Ideia é explorar porque é ruim e depois aprimorarmos

  - Abra o projeto zuul-bad
    - Explore e tente entender o que a aplicação faz
    - Que comandos o jogo aceita
    - Quantas salas estão no cenário

Exemplo

- Há cinco classes no projeto:
  - Parser
  - CommandWords
  - Command
  - Room
  - Game

  - Obs: todas as classes são bem documentadas (design ruim envolve mais do que a aparência ou documentação)

Exemplo

- Classe CommandWords
  - Define os comandos válidos no jogo
    - Array de objetos String
- Clase Parser
  - Analisador de sintaxe lê a entrada do terminal e interpreta como comandos
  - Cria objetos da classe Command que representam o comando que foi inserido

Exemplo

- Classe Command
  - Um objeto Command representa um comando que foi inserido pelo usuário
  - Tem métodos que ajudam a verificar se foi um comando válido e obter a primeira e a segunda palavra do comando como strings separadas
- Classe Room
  - Um objeto Room representa um lugar no jogo
  - As salas (Rooms) podem ter saídas que levam a outras salas

Exemplo

- Classe Game
  - É a classe principal do jogo
  - Configura o jogo e entra em loop para ler e executar comandos
  - Também contém o código que implementa cada comando do usuário

Introdução a Acoplamento e Coesão

- Acoplamento
  - Refere-se a interconectabilidade das classes (o quanto as classes estão ligadas)
  - Devemos projetar uma aplicação como um conjunto de classes cooperativas que se comunicam via interfaces bem definidas.
  - O grau de acoplamento indica a firmeza com que as classes estão conectadas
  - Buscamos um baixo grau de acoplamento ou **acoplamento fraco**

Introdução a Acoplamento e Coesão

- Ex:
  - Em uma estrutura de classes fortemente acoplada uma alteração em uma classe pode tornar necessário alterar várias outras classes

    Isso que temos que evitar!
  - Localizar todos os lugares que precisam ser modificados pode ser difícil e demorado

Introdução a Acoplamento e Coesão

- Coesão
  - Refere-se ao número e a diversidade de tarefas pelas quais uma única unidade de uma aplicação é responsável
    - Em um sistema altamente coeso uma única unidade de código (método, classe ou módulo) é responsável por uma tarefa bem definida

Queremos alto grau de coesão!

Introdução a Acoplamento e Coesão

- A razão principal por trás da coesao é a reutilização
  - Se um método ou uma classe é responsável por apenas uma coisa é mais provável que possa ser reutilizado

Prática

- Pense em um mapa para seu jogo

  (pode desenhar no papel se for preciso)
- Salve o projeto com um nome diferente
  - (por exemplo zuul)
- Altere o método createRooms() da classe Game para criar as salas e saídas que inventou
- Teste!

Prática

- Duplicação de código
  - **Indicador de design ruim!**
  - Ex: classe Game
    - Métodos printWelcome() e goRoom() contém linhas de código repetidas
    - Sintoma de má coesão
    - Problema: os dois métodos estão fazendo duas coisas e os dois imprimem informação sobre localização atual
    - **Um design aprimorado utilizaria um método mais coeso separado só para imprimir a localização atual!**
    - **Implemente um método printLocationInfo()**
    - **Teste!**

Prática

- Fazendo extensões:
  - Adicionar nova direção de movimento
  - Atualmente o jogador pode se mover em quatro direções: norte, sul, leste e oeste.
    - Queremos permitir outros níveis (ex: porões, calabouços etc)
  - Como fazer para adicionar duas novas direções? Ex: para cima e para baixo.

Prática

- Localizando código fonte relevante
  - Inspecionando o código vemos que pelo menos duas classes estão envolvidas:
    - Room e Game
  - Na classe Room
    - Saídas são listadas como campos e são atribuídas ao método setExits()
    - Para adicionar duas novas direções precisamos fazer em dois lugares

Pratica

- Na classe Game
  - Mais trabalhoso pois o código-fonte é mais extenso
  - Mantém referência a uma sala na variável currentRoom
  - Define as saídas no método createRoom
  - Imprime as saídas da sala atual no printWelcome() e no goRoom()

Prática

- Antes de fazer a modificação:
  - Acoplamento
    - O fato de ter vários lugares onde todas as saídas são enumeradas no exemplo é sintomático de design de classe ruim!
    - Imagine se quisermos adicionar outras saídas depois?!

      Opção: usar um HashMap para armazenar as saídas
    - Desenvolver um código que pode lidar com qualquer número de saídas sem tantas modificações

Pratica

- Acoplamento
  - O objeto HashMap mapeia uma direção ("north") para a sala que está nessa direção (um objeto Room)
  - Cada entrada desse HashMap terá uma string como chave e um objeto Room como valor
    - Essa alteração deve afetar a implementação (como as informações são armazenadas) da classe Room e não a interface (o que um objeto Room armazena)

Prática

- Acoplamento
  - Ideal:
    - Quando apenas a implementação da classe muda outras classes não devem ser afetadas (**acoplamento fraco**)
  - No nosso exemplo:
    - Se removermos as variáveis na classe Room substituindo por um HashMap a classe Game não compila (**acoplamento forte**)
  - Antes de introduzir o HashMap vamos desunir as classes!

Prática

- Encapsulamento
  - Um dos problemas principais é o uso de campos públicos (classe Room)

    Quebra do encapsulamento!
  - A diretriz do encapsulamento sugere que as informações sobre ***o que*** uma classe pode fazer devem ser visíveis e não ***como*** faz.
    - Podemos alterar **como** as informações são armazenadas sem prejuízo!

Prática

- Separamos *o que* de *como*, tornando os campos privados e usando métodos de acesso.
- Na classe Room:

```
class Room
{
    private String description;
    private Room northExit;
    private Room southExit;
    private Room eastExit;
    private Room westExit;

    public Room getExit(String direction){
        if (direction.equals("north"))
            return northExit;
        if (direction.equals("east"))
            return eastExit;
        if (direction.equals("south"))
            return southExit;
        if (direction.equals("west"))
            return westExit;
        return null;
    }
```

Prática

- Na classe Game:
  - Substituimos o trecho de código referente a obter a saída no goRoom por:

```
Room nextRoom = currentRoom.getExit(direction);
```

Vamos passar a responsabilidade de imprimir as saídas no formato para a classe Room, definindo um novo método:

```
/**
 * Retorna uma descricao das saidas das salas
 * por exemplo: "Exits: north west"
 */
public String getExitString()
```

Prática

```
/**
 * Retorna uma descricao das saidas da sala
 * por exemplo: "Exits: north west"
 * @return uma descricao das saidas disponiveis
 */
public String getExitString(){
    String exitString = "Exits:";
    if(northExit != null)
        exitString += "north";
    if(eastExit != null)
        exitString += "east";
    if(southExit != null)
        exitString += "south";
    if(westExit != null)
        exitString += "west";
    return exitString;
}
```

Prática

- Agora que separamos a representação interna de Room da sua interface, é facil trocar os campos por uma HashMap

```
import java.util.HashMap;
class Room
{
    private String description;
    private HashMap<String,Room> exits;

    /**
     * Create a room described "description". Initially, it has
     * no exits. "description" is something like "a kitchen" or
     * "an open court yard".
     */
    public Room(String description)
    {
        this.description = description;
        exits = new HashMap<String,Room>();
    }
```

Pratica

```
/**
 * Retorna a sala que é alcançada se sairmos desta
 * sala na direção direction. Se não houver nenhuma
 * sala nesta direção retorna nulo.
 */
public Room getExit(String direction){
    return exits.get(direction);
}
```

Prática

```java
/**
 * Define the exits of this room.  Every direction either leads
 * to another room or is null (no exit there).
 */
public void setExits(Room north, Room east, Room south, Room west)
{
    if(north != null)
        exits.put("north", north);
    if(east != null)
        exits.put("east", east);
    if(south != null)
        exits.put("south", south);
    if(west != null)
        exits.put("west", west);
}
```

Prática

- Essa modificação da classe Room agora pode ser feita sem verificar se outra coisa "quebrará" em outra parte
- Alteramos apenas aspectos privados
  - A classe está mais curta
    - Ao invés de quatro variáveis temos uma
    - Método getExit() foi simplificado
    - Incluir novas direções ficou mais fácil

Prática

- Método setExits() ainda depende das direções e precisa de aperfeiçoamento
  - É parte da interface de Room então qualquer alteração afetará outras classes (acoplamento)
- Obs:
  - Não é possível separar completamente as classes em uma aplicação senão as classes não seriam capazes de interagir entre si.
    - Tentamos manter o grau de acoplamento o mais baixo possível

Prática

- Substituimos o código de setExits() por:

```
public void setExit(String direction, Room neighbor)
{
    exits.put(direction, neighbor);
}
```

- Agora as saídas podem ser configuradas uma por vez. Na classe **Game** escrevemos:

```
lab.setExit("north", outside);
lab.setExit("east", office);
```

Prática

- Insira novas salas com novas direções:
  - "up" (para cima) e "down" (para baixo)
  - Teste!

Pratica

- Design baseado na responsabilidade
  - Cada classe deve ser responsável por tratar seus próprios dados
  - Ex:
    - Nova interface da classe Room permite a inserção de novas salas modificando o código que está apenas no método createRooms() da classe Game

```
porao = new Room("dentro do porão");
escritorio.setExit("down", porao);
porao.setExit("up", escritorio);
```

Prática

- Design baeado na responsabilidade
  - Ex:
    - Modificação para criação do método getExitString() passa a responsabilidade de impressão para a classe Room
    - As informações sobre as saídas estão armazenadas somente na própria sala

Pratica

- Obs: Como as saídas estão armazenadas em um HashMap podemos percorrer esse mapa para descrever as saídas

```
public String getExitString(){
    String returnString = "Exits: ";
    for(String exit : exits.keySet()){
        returnString += " " + exit;
    }
    return returnString;
}
```

Prática

```
public String getExitString(){
    String returnString = "Exits: ";
    for(String exit : exits.keySet()){
        returnString += " " + exit;
    }
    return returnString;
}
```

O método keySet() retorna o conjunto de chaves do hashMap (no caso as strings com os nomes das direções armazenadas para o objeto atual)

Prática

- O nosso objetivo é reduzir o acoplamento ao mínimo possível
- Ainda podemos melhorar:
  - Na classe **Game** temos uma string para descrição da sala e uma string para as saídas
  - O que acontece se adicionarmos novos itens a sala?
    - Ex: monstros, ou até outros jogadores.

Prática

- Quando descrevemos o que vemos, a lista de itens (monstros, outros jogadores) deveria ser incluída na descrição.
  - Brecha na regra de design baseado na responsabilidade
  - A classe **Room** possui as informações e deve ser responsável pela impressão
  - Qual a solução?

Prática

- Adicionar na classe **Room** o método:

```
/**
 * Retorna uma descricao longa da sala, na forma:
 *   You are in the kitchen.
 *   Exits: north west
 * @return uma descrição da sala, incluindo saídas.
 */
public String getLongDescription(){
    return "You are "+description+".\n"+getExitString();
}
```

Prática

- Na classe **Game** escrevemos:

```
System.out.println(currentRoom.getLongDescription());
```

Observação:

Futuramente para incluir novas características só precisa modificar a classe **Room**!

Prática

- Minimizando alterações
  - Idealmente apenas uma classe precisa ser alterada para fazer uma modificação
  - Ocasionalmente várias classes precisam de alteração
    - Buscamos isso com o mínimo possível de classes
    - As alterações necessárias devem ser óbvias, fáceis de detectar e fáceis de executar

Prática

- **Acoplamento implícito**
  - Definição: Uma classe depende de informações internas de outra classe mas a dependência não é tão óbvia
  - Suponha que queremos adicionar o comando look a lista de comandos válidos do jogo
    - O propósito de look é meramente imprimir a descrição da sala e sair novamente

Prática

- Para inserir um novo comando devemos adicionar ao array de palavras conhecidas validCommands na classe **CommandWords**
- Exemplo de boa coesão!
  - Em vez de definir palavras de comando no analisador de sintaxe o autor criou uma classe separada para definir palavras de comando
  - Facilita encontrar o local e é fácil acrescentar outra
  - Acrescente a palavra look e teste!

Prática

- I don't know what you mean...

Prática

- I don't know what you mean...
- A palavra foi reconhecida mas ainda não implementamos uma ação para ela.
- Adicione um método para o comando look na classe **Game**

```
private void look(){
    System.out.println(currentRoom.getLongDescription());
}
```

Prática

- Também precisamos adicionar um caso para o comando look no método processCommand:

```
String commandWord = command.getCommandWord();
if (commandWord.equals("help"))
    printHelp();
else if (commandWord.equals("go"))
    goRoom(command);
else if (commandWord.equals("look"))
    look();
else if (commandWord.equals("quit"))
    wantToQuit = quit(command);
```

Prática

- O acoplamento entre as classes Game, Parser e CommandWord até agora foi bom
  - Foi fácil fazer a extensão e funcionou rapidamente
- Problema:
  - Acoplamento implícito torna-se aparente quando emitimos um comando 'help'
  - Teste! O que acontece?

Prática

- O texto de ajuda está incompleto
  - O novo comando look, não é listado!
- Se não tivessemos notado esse problema?
- Cada vez que adicionarmos um comando, o texto da ajuda precisa ser modificado
  - E isso é fácil de esquecer!
- O programa compila e executa normalmente.

Prática

- **Acoplamento implícito:**
  - Quando comandos são alterados o texto de ajuda deve ser modificado (**acoplamento**)
  - Mas nada no fonte do programa indica claramente essa dependência (**implícito**)
  - Uma classe bem projetada evitará essa forma de acoplamento seguindo a regra do design baseado na responsabilidade:
    - Como a classe CommandWord é responsável or palavras de comando, também deve ser responsável pela sua impressão

Prática

- Adicionamos o método a classe **CommandWord**:

```
/**
 * Imprimir todos os comandos válidos para System.
 */
public void showAll(){
    for (String command : validCommands){
        System.out.println(command + " ");
    }
    System.out.println();
}
```

Ao invés de imprimir um texto fixo, printHelp() agora chama o método showAll() para solicitar que a classe CommandWords faça isso.

Prática

- Observe o diagrama de classes.
- A classe Game não tem conhecimento da classe CommandWords (veja as setas!)
  - Obs: quanto menor o número de setas menor o acoplamento
- A classe Game tem acesso a classe Parser e não vamos modificar aumentando o acoplamento, ao invés disso vamos usar o parser para imprimir a ajuda.

Prática

- Na classe Game modificamos o método printHelp():

```
System.out.println("Suas palavras de ajuda são");
Parser.showCommands();
```

- Falta o método showCommands() na classe Parser:

```
/**
 * Imprime uma lista de palavras de comando validas
 */
public void showCommands(){
    commands.showAll();
}
```

Prática

- Teste.

- Se quiser inserir um novo comando, será preciso alterar a classe **Game**?
  - Ex: comando **eat** (comer)

    Ação: Imprime na tela: "Você comeu e não está mais com fome."

Prática

- Pensando à frente
  - Pode ser uma extensão interessante adicionar uma interface gráfica com o usuário com menus, botões e imagens, nesse caso não queremos mais imprimir informações no terminal
  - Seria uma boa ideia encapsular as informações sobre interface com o usuário em uma ou um conjunto de classes bem definido
  - Nossa alteração da impressão dos comandos não segue essa regra de design

Prática

- Pensando à frente:
  - Seria mais apropriado definir que CommandWords é responsável por produzir a lista de palavras de comando e a classe Game decide como será apresentada ao usuário.
  - Alcançamos isso alterando o método **showAll**() de modo que retorne uma string com as palavras de comandos (renomeando para **getCommandList**())
  - Obs: não traz beneficio agora mas podemos nos beneficiar desse design no futuro

Coesão

Prática

- Coesão de métodos
  - Todo método deve ser responsável somente por uma tarefa bem definida
  - Ex:
    - Método printWelcome() da classe **Game**

Prática

- Coesão de classes
  - Cada classe deve representar uma única entidade bem-definida no domínio do problema
  - Como exemplo de coesão de classes, vamos adicionar itens ao jogo
    - Cada sala pode manter um item e um item possui descrição e peso
    - O peso poderá ser usado mais tarde para determinar se pode ser coletado ou não

Prática

- Coesão de classes

  Soluções:

  - Adicionar campos itemDescricao e itemPeso na classe Room

Prática

- Coesão de classes

  Soluções:

  - Adicionar campos itemDescricao e itemPeso na classe Room
    - Grau de coesão ruim porque Room descreve tanto sala quanto item
    - Item vinculado a uma sala especifica

Prática

- Coesão de Classes:
  - Um design aprimorado criaria uma classe separada para itens (chamada Item)
  - Faça essa modificação!
    - Uma sala contem um único item
    - Os itens tem descricao e peso
    - Quando criar salas e configurar saidas os itens também devem ser criados
    - Quando um jogador entra em uma sala as informações sobre um item presente também deve ser exibidas

Prática

- E se quiser permitir um número ilimitado de itens?

Prática

- E se quiser permitir um número ilimitado de itens?
- Utilizando uma classe separada para itens fica fácil
  - Criamos vários objetos Item e armazenamos em uma coleção na sala (Room)

Coesão para legibilidade e coesão para reutilização

- Existem varias maneiras de a coesão elevada beneficiar o projeto
- As duas mais importantes são
  - Legibilidade e reutilização
- Ex:
  - A coesão do método printWelcome() é claramente um exemplo que torna uma classe mais legível, mais fácil de entender

Coesão para Legibilidade

- Ex:
  - Classe Item separada é um exemplo de coesão de classe com elemento de legibilidade
  - Um programador de manutenção facilmente reconhecerá onde iniciar a leiura do código se uma alteração para as características de um item for necessária
  - A coesão de classes também aumenta a legibilidade de um programa

Coesão para reutilização

- Ex:
  - A coesão da classe item também facilita reutilização
  - Podemos criar múltiplos itens e utilizar o mesmo código em mais de um item

Coesão para reutilização

- Ex:
  - Também é um aspecto importante de coesão de método
  - Considere um método na classe Room com a assinatura:

```
public Room leaveRoom(String direction)
```

Coesão para reutilização

- Ex:
  - Este método poderia retornar a sala na direção dada (de modo que seria utilizada como nova currentRoom)
  - E também imprime a descrição da nova sala
  - Em nossa versão separamos essa tarefa em dois métodos:

```
public Room getExit(String direction)
public String getLongDescription()
```

Coesão para reutilização

- A vantagem desse projeto é que as tarefas separads podem ser reutilizadas mais facilmente
- O método getLongDescription() por exemplo, é utilizado nos métodos:
  - goRoom()
  - printWelcome()
  - Look()
- Só é possível pelo alto grau de coesão!

Desafio

- Implemente um comando 'back'
  - Esse comando não tem uma segunda palavra
  - Leva o jogador para a última sala que esteve
  - Permita que utilizando várias vezes faça voltar várias salas até o início do jogo se for preciso
    - Dica: utilize um Stack